



# SYSMON HUNTER

Detection Engineering & Analyst Console

---

User Manual

## Version 0.3.4

Real-time Sysmon detection, correlation, and incident triage



# Contents

---

## 1. Introduction

## 2. System Architecture

### 2.1 Event Pipeline

### 2.2 Data Flow

## 3. Detection Engine

### 3.1 Rule-Based Detection

### 3.2 Process-Tree Correlation

### 3.3 Statistical Beacon Detection

### 3.4 Reconnaissance-Burst Detection

### 3.5 Ransomware Detection

### 3.6 Behavior Profiling & Derived Titles

### 3.7 Incident Scoring

### 3.9 Sigma Rule Import

### 3.10 STIX 2.1 Export

### 3.11 ATT&CK Coverage Report & Navigator Export

### 3.12 False-Positive Similarity

## 4. Detection Rule Catalog

## 5. The Analyst Console

### 5.1 Incident Queue

### 5.2 Incident Detail Views

### 5.3 Explore View

### 5.4 Search

### 5.5 IOC Enrichment

### 5.6 ATT&CK Technique Reference

### 5.7 Analyst Notes

### 5.8 PDF Reports & STIX Export

### 5.9 Incident Triage

### 5.10 Sigma Rule Import

### 5.11 Theme & Database Reset

### 5.12 Live WebSocket Feed

### 5.13 ATT&CK Coverage Download

### 5.14 Stats Dashboard

## 6. REST API Reference

## 7. Configuration Reference

## 8. Installation & Getting Started

## 9. Testing

## 10. Docker Deployment

## 11. Design Decisions

## 12. Project Layout



# 1. Introduction

Sysmon Hunter is a real-time detection and correlation engine for Windows Sysmon telemetry, paired with a live analyst console. It ingests events from an endpoint, matches them against ATT&CK-mapped detection rules, reconstructs the process tree to correlate related detections into incidents, detects command-and-control beaconing and ransomware activity statistically, and enriches indicators against external reputation sources -- all streamed to the console in real time.

The project's guiding principle is that a single matched rule is a lead, not a verdict. A lone "PowerShell ran an encoded command" event is worth a glance. The same detection sitting under a Microsoft Word process, next to a reconnaissance burst and an outbound beacon, with a file hash confirmed malicious by VirusTotal, is an incident an analyst can act on immediately. Everything documented in this manual serves that one idea: turning individually weak signals into a correlated, prioritized, investigable incident.

This manual documents the detection engine, the analyst console, the REST and WebSocket API, the configuration surface, and the deployment options as of the version printed on the cover page.

## Who this is for

Detection engineers writing or tuning Sysmon-based rules; SOC analysts triaging incidents in the console; and anyone standing up the engine against their own endpoint telemetry or a corpus of recorded Sysmon events.

# 2. System Architecture

## 2.1 Event Pipeline

Every event, regardless of source -- a live Winlogbeat feed, a replayed .evtx file, or a seed script -- takes the same path through the engine. The pipeline is independent of HTTP: it runs identically under the `/ingest` endpoint, the EVTXX replay script, and the automated test suite with no server running at all.

| Step | Stage               | What happens                                                                                                                                                               |
|------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Ingest              | POST <code>/ingest</code> receives one normalized event and hands it to the pipeline. Holds no logic of its own.                                                           |
| 2    | Normalize           | Winlogbeat/Sysmon JSON is converted into a uniform internal Event, so downstream code never branches on source format.                                                     |
| 3    | Observe             | Every event feeds the in-memory process tree, not just the ones that match a rule -- a malicious process's ancestors are usually benign and must still be recorded.        |
| 4    | Detect              | The event is run against four parallel detectors: the YAML rule engine, the statistical beacon detector, the reconnaissance-burst detector, and the network-scan detector. |
| 5    | Correlate           | New detections are grouped with existing ones that share a process-tree root within the correlation window, forming or extending an incident.                              |
| 6    | Persist & Broadcast | The detection and incident are written to SQLite and pushed to every connected console over the WebSocket feed.                                                            |



## 2.2 Data Flow

The process tree is keyed on Sysmon's ProcessGuid, never on process ID. Windows recycles PIDs aggressively, so a tree keyed on PID would graft an unrelated later process onto a malicious parent's identity the moment that PID is reused; ProcessGuid is unique across reboots and PID reuse and does not have this failure mode.

Incidents are not simply a rule match logged to a table -- the correlator groups every detection that shares a process-tree root within a configurable time window into a single incident, and carries the full branching tree with it, not just the linear chain from root to the triggering process.

## 3. Detection Engine

### 3.1 Rule-Based Detection

157 YAML detection rules, each mapped to one or more MITRE ATT&CK technique IDs, are indexed by Sysmon Event ID so that only relevant rules are evaluated per event. Rules use Sigma-compatible matching semantics: field/operator pairs (equals, contains, startswith, endswith, re) combined with an all/any condition, and an operator's expected value may be a single value or a list, OR'd together -- for example, matching a file write against any of several known ransomware extensions in one clause.

The full catalog, with severity and ATT&CK mapping for every rule, is in Section 4.

### 3.2 Process-Tree Correlation

Detections rarely stand alone in a real intrusion. The correlator reconstructs each host's process ancestry from Sysmon's ProcessGuid fields, and any detections whose processes share a tree root within the correlation window (10 minutes by default) are grouped into one incident carrying the complete branching tree -- not just the chain from root to the flagged process. A foothold that spawned several children shows every branch, whether or not each child individually fired a rule.

### 3.3 Statistical Beacon Detection

No single network-connection event can reveal a periodic command-and-control channel; it takes a sequence of them. The beacon detector watches outbound connections (Sysmon Event ID 3) per process and scores their timing for regularity, requiring a minimum number of callbacks before rendering a verdict. It uses the median and median absolute deviation of the intervals, not the mean and standard deviation -- a live C2 session produces occasional large outliers (the operator interacts, a callback retries late), and those outliers distort a standard-deviation score badly enough to lose the channel entirely, while the median barely moves. Jitter up to and beyond Cobalt Strike's default 37% is still caught. A configurable exclusion list keeps common legitimately-periodic processes (browsers, sync clients, chat apps) from generating noise.

### 3.4 Reconnaissance-Burst Detection

The discovery detector counts *distinct* ATT&CK discovery techniques observed in one process tree within a short window, not raw execution counts. A script re-running systeminfo in a loop is volume without variety and stays quiet; an operator running whoami, net, nltest, and systeminfo in quick succession is variety, and is flagged as a reconnaissance burst.



### 3.5 Statistical Network-Scan Detection

Beaconing looks at rhythm to one destination; the discovery burst looks at command variety. Neither sees a scanner, because a scanner is neither periodic nor does it run recon commands -- it simply opens many connections to many different places, fast. The scan detector watches outbound connections (Sysmon Event ID 3) per process and tracks the growing set of distinct destinations it has touched within a window, flagging breadth rather than timing. Two shapes are covered, either sufficient on its own: many distinct ports against a small number of hosts (a port scan, the textbook vertical case) and many distinct hosts on a small number of ports (a host sweep, the horizontal case, e.g. an SMB or RDP sweep across a subnet). Severity escalates when a later alert's accumulated breadth passes double the configured threshold, and ATT&CK tagging reflects whichever shape -- or both -- was actually observed.

### 3.6 Ransomware Detection

Two rules target the file-write behavior that precedes and accompanies a ransomware encryption event, in addition to the shadow-copy-deletion and recovery-disabling command-line rules that typically precede it: a ransom note dropped to disk, matched against the near-universal "how to decrypt / restore your files" naming convention used across ransomware families, and a mass file write ending in a known ransomware encryption extension (.locked, .encrypted, .crypt, .wcry, and others). Because these detections have no process command line to show as evidence -- the interesting fact is which file was written, not what process wrote it -- the console surfaces the exact matched file path everywhere the detection's detail is shown: the incident's detection list, its timeline and process-tree popups, and the full-screen tree viewer.

### 3.7 Behavior Profiling & Derived Titles

Rather than leaving an analyst to read a flat list of N detections, the profiling engine turns an incident's detections into a plain-language, kill-chain-ordered narrative -- for example: "gained initial access through a phishing document, executed an obfuscated PowerShell payload, harvested credentials from LSASS, beacons to C2 every ~35 seconds." Each incident is also given a derived title summarizing its contents at a glance in the queue -- "Phishing to reconnaissance," "Credential access with C2," "Ransomware preparation" -- instead of a generic "Incident #4821."

### 3.8 Incident Scoring

Incident severity scoring is non-linear by design: one critical LSASS access outweighs three medium-severity suspicious-path executions, and two high-severity detections together read as critical. This mirrors how an analyst actually judges risk -- a single strong signal can outweigh several weak ones, and enough weak signals together compound into a strong one. An incident is promoted to "active" once its cumulative score crosses a configurable threshold (12, by default).



### 3.9 Correlation Chains & Classification

Beyond the tactic-based titles in Section 3.6, three named, rule-ID-level patterns recognise a specific multi-stage story and outrank the generic titles when they match: a **ransomware activity chain** (shadow-copy deletion together with a ransom note or an encrypted-extension write), a **credential-theft campaign** (at least two distinct credential-access rules -- LSASS access, Mimikatz, DCSync, browser/KeePass credential-database staging, Kerberoasting, or AS-REP roasting -- firing on the same incident), and an **Office-to-PowerShell infection chain** (an Office application spawning a shell, followed by a PowerShell download cradle). A matched chain sets both the incident's title and a machine-readable classification field, shown as a badge in the console queue and on the incident's full page. This is deliberately an identity mechanism, not a scoring one -- every rule referenced by a chain is itself high or critical severity, so a matched chain has already cleared the actionable threshold on the ordinary scoring scale described in Section 3.7 before it is ever classified.

### 3.10 Sigma Rule Import

The hand-written rule catalog in Section 4 can be extended at runtime by uploading Sigma YAML files (from the public SigmaHQ corpus or written by hand) through the console's settings menu, which posts to `POST /admin/rules/import-sigma`. Each document is converted independently by `backend/engine/sigma_import.py` into this engine's own rule schema and written under `rules/imported_sigma/`, then the whole rule store reloads from disk -- an import is live for the very next ingested event, no restart required.

Sigma is a considerably richer language than the matcher in Section 3.1 implements, so the importer supports a deliberately bounded subset: Windows Sysmon logsource categories mapped to the matching EventID, a single selection or several combined with a plain and / or / 1 of `x*` / all of `x*` condition, one trailing and not `<filter>` exclusion, the contains / startswith / endswith / re modifiers, and automatic translation of bare glob wildcards (`*`, `?`) into the matching operator. A Sigma rule that needs nested boolean groups, an aggregation (`count()`, `near`), or a modifier this matcher cannot reproduce (`|all`, `|base64`, `|cidr`, numeric comparisons) is rejected with the specific reason rather than approximated -- the same policy the rule loader already applies to a malformed YAML file on disk: one bad rule is reported and skipped, never silently wrong and never fatal to the batch.

### 3.11 STIX 2.1 Export

The counterpart to Sigma import: any incident can be downloaded as a STIX 2.1 bundle from `GET /incidents/{id}/stix` (or the "Download STIX bundle" button next to the PDF report button), for another threat-intel platform to ingest without ever needing to know this engine's own schema. `backend/engine/stix_export.py` builds an identity object for this engine, one attack-pattern per distinct ATT&CK technique the incident's detections carry, one indicator per pivotable IOC already surfaced by the key-indicators consolidation in Section 5 (a C2 destination IP or domain, a file's SHA256, a persistence registry key -- deduplicated across detections, never one indicator per detection), and a report object whose `object_refs` ties all of it together, named and described from the incident's own title and behavior-profile narrative.

Object identifiers are deterministic -- a UUIDv5 seeded on stable content (a technique ID, an indicator's own pattern string, the incident ID) rather than randomly generated -- so exporting the same incident twice, or the same technique across two different incidents, produces the same STIX ID both times. That is what lets a receiving platform de-duplicate objects across imports instead of accumulating a new copy of the same technique on every download. No relationship objects are generated between indicators and techniques: the underlying data is incident-level, not per-IOC, so asserting that a specific indicator indicates



a specific technique would claim precision the source data does not have.

### 3.12 ATT&CK Coverage Report & Navigator Export

Every other view in this manual answers what the engine has detected; this one answers what it never could. `GET /attack/coverage` counts how many rules and statistical detectors raise each ATT&CK technique across the whole corpus, and `GET /attack/coverage/navigator` (also reachable from the settings menu as “Download ATT&CK coverage”) renders that count as a MITRE ATT&CK Navigator layer -- open it at the public Navigator and every technique is colored red through green by how many rules cover it, turning months of reactive rule-writing into a single prioritized worklist for the next rule.

The report has two honesty levels, and it says which one it is giving you. `backend/data/attack_data.json` (Section 5.6) only ever contains techniques this project already references, by design -- so on its own it can rank covered techniques against each other but can never show a technique with zero rules, since that technique was filtered out before the report ever runs. `scripts/fetch_attack.py` also writes a second, full-catalog file, `backend/data/attack_index.json` -- every non-deprecated Enterprise technique, covered or not, with just enough data (ID, name, tactics) to plot on a layer. When that file is present, `backend/engine/coverage.py` produces a true gap analysis; when it is absent, the report still runs, it is just explicitly marked `partial` in both the JSON response and the layer's own description -- a missing optional file changes how much the report can show, never whether it works.

### 3.13 False-Positive Similarity

Every incident an analyst marks a false positive is a labeled example of noise -- `backend/engine/noise.py` is what makes that marking pay off for the next incident, without a trained model or a mountain of data to get there. A newly-opened incident is compared against the deployment's own history of confirmed false positives on four explainable signals: which detection rules fired (the most precise statement of “this is the same pattern” available, weighted highest), which ATT&CK techniques were involved, whether the same process was the root of the tree, and how much of the process chain overlaps (weighted lowest, since two unrelated incidents can share common ancestor processes like `explorer.exe` by platform convention alone).

This is deliberately not a classifier. It produces a score -- e.g. “78% similar to incident abc123, because: same detection rules, same root process” -- from the very first false positive an analyst ever marks, and the explanation is exactly the weighted arithmetic that produced it, nothing hidden and nothing to retrain. A match at or above `HUNTER_NOISE_SIMILARITY_THRESHOLD` (default 0.6, Section 7) surfaces as a dashed “probable noise” badge in the console queue and on the incident page, naming the past incident it resembles and the specific signals that matched -- a hint for an analyst to weigh, never an automatic dismissal. Only open incidents are scored; a closed or already-dismissed incident has nothing left to warn about.



## 4. Detection Rule Catalog

Every rule below was written and validated against real telemetry -- either a hand-built scenario exercised end-to-end through the console, or a known-malicious .evtx sample from a public detection-engineering corpus. Each has an automated true-positive case (the rule fires on the malicious sample) and a true-negative case (it stays quiet on adjacent legitimate activity) in the test suite.

| Rule ID | Event              | Severity | Title                                                        | ATT&CK                      |
|---------|--------------------|----------|--------------------------------------------------------------|-----------------------------|
| SYS-001 | Process Create     | High     | Office application spawned a command interpreter             | T1566.001, T1059            |
| SYS-002 | Process Create     | High     | PowerShell executed an encoded command                       | T1059.001, T1027            |
| SYS-003 | Process Create     | High     | LOLBin used to download a remote payload                     | T1105, T1218                |
| SYS-004 | Process Create     | Critical | Volume shadow copies deleted                                 | T1490                       |
| SYS-005 | Process Create     | High     | Script engine spawned from a browser or mail client          | T1204.002, T1566            |
| SYS-006 | Process Create     | Medium   | Executable ran from a user-writable staging path             | T1204                       |
| SYS-007 | Process Create     | High     | System binary running from the wrong path                    | T1036.005                   |
| SYS-008 | Process Create     | Medium   | rundll32 executed without a DLL argument                     | T1218.011                   |
| SYS-009 | Process Create     | High     | PowerShell download cradle                                   | T1059.001, T1105            |
| SYS-010 | Process Access     | Critical | Suspicious process accessed LSASS memory                     | T1003.001                   |
| SYS-020 | Network Connection | Medium   | Scripting or LOLBin process made an outbound connection      | T1071                       |
| SYS-021 | Network Connection | Medium   | Connection to a common C2 port from a non-browser process    | T1571                       |
| SYS-030 | Registry Set       | High     | Autorun registry key modified                                | T1547.001                   |
| SYS-031 | Registry Set       | High     | Windows Defender setting disabled via registry               | T1685                       |
| SYS-032 | Registry Set       | High     | Image File Execution Options debugger set                    | T1546.012                   |
| SYS-034 | Process Create     | High     | Control panel item (.cpl) executed from a user-writable path | T1218.002                   |
| SYS-035 | Process Create     | High     | Script host executing an encoded script                      | T1059.005, T1059.007, T1027 |
| SYS-036 | Process Create     | High     | Script host spawned by rundll32                              | T1059, T1218.011            |
| SYS-037 | Process Create     | High     | rundll32 invoking a known LOLBAS DLL export                  | T1218.011, T1204.001        |
| SYS-038 | Process Create     | High     | IIS credentials extracted via appcmd                         | T1552.001, T1003            |
| SYS-039 | Process Create     | Medium   | appcmd executed by a web server worker or script host        | T1552.001                   |
| SYS-040 | Image Load         | High     | PowerShell engine loaded by a non-PowerShell process         | T1059.001                   |
| SYS-041 | Process Access     | Critical | LSASS opened with credential-dumping access rights           | T1003.001                   |
| SYS-050 | File Create        | High     | File dropped into a Startup folder                           | T1547.001                   |
| SYS-051 | File Create        | High     | Office application wrote a script or executable              | T1566.001                   |
| SYS-060 | Pipe Created       | High     | Named pipe matching a known C2 default                       | T1071, T1055                |
| SYS-070 | Process Create     | Medium   | WMI persistence staged via mofcomp or wmic                   | T1546.003                   |
| SYS-071 | WMI Event          | High     | WMI event consumer registered for code execution             | T1546.003                   |





| Rule ID | Event                  | Severity | Title                                                                    | ATT&CK               |
|---------|------------------------|----------|--------------------------------------------------------------------------|----------------------|
| SYS-072 | Pipe Created           | High     | Named pipe matching PsExec-style remote execution                        | T1021.002, T1569.002 |
| SYS-073 | Process Create         | High     | PsExec service binary executed on this host                              | T1569.002, T1021.002 |
| SYS-074 | Process Create         | High     | Regsvr32 registering a scriptlet via scrobj.dll (Squiblydoo)             | T1218.010            |
| SYS-075 | Process Create         | Medium   | Certutil used to decode or encode a local file                           | T1140                |
| SYS-076 | Process Create         | Medium   | BITS job created via the Start-BitsTransfer cmdlet                       | T1197                |
| SYS-077 | Driver Load            | High     | Unsigned kernel driver loaded                                            | T1211, T1562.001     |
| SYS-078 | Registry Set           | High     | Registry Run key written by the WMI provider host                        | T1047, T1547.001     |
| SYS-079 | Process Create         | High     | FTP client spawned a child process                                       | T1105, T1202         |
| SYS-080 | File Create            | Critical | Ransom note dropped                                                      | T1486, T1491.001     |
| SYS-081 | File Create            | Critical | File written with a known ransomware encryption extension                | T1486                |
| SYS-082 | Process Create         | Critical | Credential dumping via comsvcs.dll MiniDump export                       | T1003.001            |
| SYS-083 | Registry Set           | High     | Fileless UAC bypass via auto-elevate registry hijack                     | T1548.002            |
| SYS-084 | Process Create         | High     | cmstp executed with a silent or auto-install flag                        | T1218.003            |
| SYS-085 | Registry Set           | High     | Port forwarding rule added via netsh                                     | T1090.001            |
| SYS-086 | Registry Set           | High     | PowerShell script block logging disabled via registry                    | T1562.001            |
| SYS-087 | Registry Create/Delete | High     | PowerShell Constrained Language Mode lockdown policy removed             | T1562.001            |
| SYS-088 | Process Create         | Critical | IIS worker process spawned a command shell                               | T1505.003            |
| SYS-089 | Process Create         | Critical | SQL Server process spawned a command shell                               | T1059.003, T1190     |
| SYS-090 | Process Create         | Medium   | Process spawned by the PowerShell remoting host                          | T1021.006            |
| SYS-091 | Registry Set           | Medium   | Local account or administrators-group membership changed in the SAM hive | T1136.001, T1098     |
| SYS-092 | Process Create         | High     | PE metadata claims a trusted binary, but it runs from a staging path     | T1036.005            |
| SYS-093 | Process Create         | High     | Scheduled task created via schtasks                                      | T1053.005            |
| SYS-094 | Registry Set           | High     | New service image path points at a user-writable staging location        | T1543.003            |
| SYS-095 | Remote Thread Created  | Critical | Remote thread created inside LSASS                                       | T1055, T1003.001     |
| SYS-096 | Process Create         | High     | Firewall rule added to allow inbound traffic                             | T1562.004            |
| SYS-097 | Process Create         | High     | Windows event log cleared via wevtutil                                   | T1070.001            |
| SYS-098 | Process Create         | Medium   | Archive utility invoked with a password, staging data for exfiltration   | T1560.001            |
| SYS-099 | Process Create         | High     | .NET installer utility executed from a user-writable staging path        | T1218.004, T1218.009 |
| SYS-100 | Process Create         | High     | mshta executed a remote HTA                                              | T1218.005            |
| SYS-101 | Process Create         | Medium   | Compiled HTML Help opened from a user-writable staging path              | T1218.001            |



| Rule ID | Event          | Severity | Title                                                                 | ATT&CK               |
|---------|----------------|----------|-----------------------------------------------------------------------|----------------------|
| SYS-102 | Process Create | High     | msiexec installed a package from a remote URL                         | T1218.007            |
| SYS-103 | Process Create | Medium   | Account added to the local Administrators group via net.exe           | T1098, T1136.001     |
| SYS-104 | Process Create | High     | A security or update service was stopped from the command line        | T1489, T1562.001     |
| SYS-105 | Process Create | High     | Sysinternals sdelete executed                                         | T1070.004            |
| SYS-106 | Process Create | Medium   | cipher used to wipe free disk space                                   | T1070.004            |
| SYS-107 | Registry Set   | Medium   | Remote Desktop enabled via registry                                   | T1021.001            |
| SYS-108 | Process Create | Critical | Credential hive saved to disk via reg.exe                             | T1003.002            |
| SYS-109 | Process Create | Critical | Sysinternals procdump run against LSASS                               | T1003.001            |
| SYS-110 | Process Create | Critical | NTDS.dit extracted via ntdsutil                                       | T1003.003            |
| SYS-111 | Process Create | High     | mavinject used to inject into a running process                       | T1055.001            |
| SYS-112 | Process Create | Medium   | WSL used to execute a command outside its Linux filesystem            | T1202, T1059.004     |
| SYS-113 | Process Create | High     | MSBuild ran a project file staged in a user-writable path             | T1127.001            |
| SYS-114 | Process Create | Medium   | Uncommon signed proxy-execution binary run against a staged target    | T1218, T1216         |
| SYS-115 | Process Create | High     | RDP session hijacked via tscon                                        | T1563.002            |
| SYS-116 | Process Create | High     | Active Directory reconnaissance tool executed                         | T1087, T1482         |
| SYS-117 | Process Create | Medium   | Cloud sync/transfer tool executed                                     | T1567.002            |
| SYS-118 | Process Create | High     | Kerberoasting tooling executed                                        | T1558.003            |
| SYS-119 | Process Create | High     | PowerShell launched in version-2 downgrade mode                       | T1059.001            |
| SYS-120 | Process Create | High     | Command line references a known AMSI-bypass signature                 | T1562.001            |
| SYS-121 | Process Create | Medium   | bitsadmin used to transfer a file                                     | T1197                |
| SYS-122 | Registry Set   | High     | CLSID InprocServer32 handler registered under a user's own hive       | T1546.015            |
| SYS-123 | Process Create | High     | Command line references a known privilege-escalation token-theft tool | T1134                |
| SYS-124 | 22             | Medium   | DNS query to a dynamic DNS domain                                     | T1071.004, T1568     |
| SYS-125 | 23             | Medium   | Executable deleted from a staging path shortly after being dropped    | T1070.004            |
| SYS-126 | 25             | Critical | Process image tampering detected (hollowing or doppelganging)         | T1055.012, T1055.013 |
| SYS-127 | 15             | High     | Executable content staged inside an NTFS alternate data stream        | T1564.004            |
| SYS-128 | 9              | High     | Raw volume access by a process outside known disk utilities           | T1006                |
| SYS-129 | 2              | Medium   | File creation timestamp changed outside a servicing process           | T1070.006            |
| SYS-130 | Process Create | Critical | DCSync-style directory replication requested from a command line      | T1003.006            |



| Rule ID | Event          | Severity | Title                                                              | ATT&CK    |
|---------|----------------|----------|--------------------------------------------------------------------|-----------|
| SYS-131 | Process Create | Critical | Command line references a known Mimikatz module or command         | T1003.001 |
| SYS-132 | Process Create | High     | Odbcconf used as a signed proxy-execution binary                   | T1218.008 |
| SYS-133 | Process Create | High     | mmc.exe loaded a Management Saved Console from a staging path      | T1218.014 |
| SYS-135 | File Create    | High     | Non-browser process touched a browser credential database          | T1555.003 |
| SYS-136 | File Create    | High     | Script interpreter touched a KeePass database                      | T1555     |
| SYS-137 | File Create    | High     | Browser credential database written into a staging directory       | T1555.003 |
| SYS-138 | Process Create | Medium   | Network configuration discovery command executed                   | T1016     |
| SYS-139 | Process Create | Medium   | Active network connections enumerated via netstat                  | T1049     |
| SYS-140 | Process Create | Medium   | Recursive filesystem enumeration executed                          | T1083     |
| SYS-141 | Process Create | High     | Security or EDR product enumerated from the command line           | T1518.001 |
| SYS-142 | Process Create | Medium   | Archive created with its header/file-list encrypted                | T1560.001 |
| SYS-143 | Process Create | High     | Rclone ran a copy/sync against a remote destination                | T1567.002 |
| SYS-144 | Process Create | High     | Command-line tooling uploaded a file to a remote server            | T1048     |
| SYS-148 | Process Create | High     | SharpHound invoked via script or named by its collection arguments | T1482     |
| SYS-149 | Process Create | Medium   | Active Directory account or group enumeration command executed     | T1087     |
| SYS-150 | Process Create | High     | AS-REP roasting tooling executed                                   | T1558.004 |
| SYS-151 | Process Create | High     | ClickFix/FileFix decoy verification lure pasted from Explorer      | T1204.004 |
| SYS-152 | Process Create | High     | PowerShell launched from Explorer with bypass + hidden window      | T1204.004 |
| SYS-153 | Process Create | High     | Script host from Explorer immediately fetching remote code         | T1204.004 |
| SYS-154 | Process Create | High     | PowerShell reads and executes clipboard contents                   | T1204.004 |
| SYS-155 | Process Create | High     | Remote-access/RMM software launched from a non-Explorer parent     | T1219     |
| SYS-156 | Process Create | High     | Remote-access/RMM software installed with a silent flag            | T1219     |
| SYS-157 | Process Create | High     | PowerShell reads and overwrites the clipboard (crypto-clipper)     | T1115     |
| SYS-158 | Process Create | High     | Problem Steps Recorder invoked for silent screen capture           | T1113     |
| SYS-159 | Process Create | High     | Chat-service webhook or bot API used as a covert channel           | T1102     |
| SYS-160 | Process Create | High     | Public paste service used as a C2 dead-drop                        | T1102     |
| SYS-161 | Process Create | Medium   | Archive utility packaged a broad user data directory               | T1074     |



| Rule ID | Event          | Severity | Title                                                                | ATT&CK               |
|---------|----------------|----------|----------------------------------------------------------------------|----------------------|
| SYS-162 | Process Create | Medium   | robocopy mass-mirrored a broad user directory tree                   | T1119, T1074         |
| SYS-163 | Process Create | Critical | Rubeus used to pass or renew a Kerberos ticket                       | T1550.003            |
| SYS-164 | Process Create | Critical | Mimikatz used to pass a hash or a Kerberos ticket                    | T1550.002, T1550.003 |
| SYS-165 | Process Create | Critical | Known Group Policy abuse tooling or cmdlet invoked                   | T1484.001            |
| SYS-166 | Process Create | Critical | Domain trust modified to disable SID-filtering protections           | T1484.002            |
| SYS-167 | Process Create | Critical | Command-line destructive wipe of a broad user directory or volume    | T1485                |
| SYS-168 | Process Create | Critical | PowerShell bulk-disabled or reset a broad set of AD accounts         | T1531                |
| SYS-169 | File Create    | Critical | Script interpreter staged a cloud CLI credential file                | T1552.001            |
| SYS-170 | Process Create | Critical | Cloud instance metadata service queried by a non-agent process       | T1552.005            |
| SYS-171 | File Create    | High     | Script interpreter staged an SSH or PuTTY private key                | T1552.004            |
| SYS-172 | Process Create | High     | WMI used to create a process via Win32_Process                       | T1047                |
| SYS-173 | Process Create | High     | mmc.exe spawned a shell consistent with MMC20.Application DCOM abuse | T1021.003            |
| SYS-174 | Process Create | High     | Docker container launched with --privileged                          | T1611                |
| SYS-175 | File Create    | Critical | Accessibility binary replaced on disk                                | T1546.008            |
| SYS-176 | Registry Set   | High     | Winlogon helper DLL persistence key set                              | T1547.004            |
| SYS-177 | Registry Set   | High     | Security Support Provider registered                                 | T1547.005            |
| SYS-178 | Registry Set   | High     | Active Setup StubPath persistence key set                            | T1547.014            |
| SYS-179 | Process Create | High     | Domain account created                                               | T1136.002            |
| SYS-180 | Process Create | Critical | Boot configured into Safe Mode                                       | T1562.009            |
| SYS-181 | Process Create | High     | Windows Event Log service or channel disabled                        | T1562.002            |
| SYS-182 | Process Create | High     | Disk image mounted via PowerShell                                    | T1204.003            |
| SYS-183 | Process Create | Critical | Mimikatz used to dump LSA secrets or cached domain credentials       | T1003.004/.005       |
| SYS-184 | Process Create | Critical | Mimikatz used to forge a Kerberos golden ticket                      | T1558.001            |
| SYS-185 | File Create    | High     | File written with a double extension masking an executable           | T1036.007            |
| SYS-186 | Process Create | High     | Registry queried for an autologon or default password                | T1552.002            |
| SYS-187 | Process Create | Critical | Group Policy Preferences cpassword harvested                         | T1552.006            |
| SYS-188 | Process Create | High     | SSH client used with a private key for lateral movement              | T1021.004            |
| SYS-189 | Process Create | High     | Tor or a multi-hop proxy tool launched                               | T1090.003            |
| SYS-190 | Process Create | High     | PowerShell archived data via .NET compression                        | T1560.002            |
| SYS-191 | 16             | Critical | Sysmon configuration reloaded                                        | T1562.001/.006       |
| SYS-192 | 19             | High     | WMI event filter registered                                          | T1546.003            |
| SYS-193 | 21             | High     | WMI filter bound to a consumer                                       | T1546.003            |



| Rule ID | Event          | Severity | Title                                                                       | ATT&CK                          |
|---------|----------------|----------|-----------------------------------------------------------------------------|---------------------------------|
| SYS-194 | 18             | High     | Named pipe connected matching a known C2/lateral-movement signature         | T1071/T1055/T1021.002/T1569.002 |
| SYS-195 | 24             | Medium   | Clipboard accessed by a scripting engine or LOLBIN                          | T1115                           |
| SYS-196 | 14             | High     | Persistence-relevant registry key renamed                                   | T1112/T1564.001                 |
| SYS-197 | Image Load     | High     | Unsigned DLL loaded from a user-writable path by a trusted-location process | T1574.001/.002                  |
| SYS-198 | Process Create | High     | Certutil used to install a certificate into a trust store                   | T1553.004                       |
| SYS-199 | Process Create | High     | Executable disguised as an installer package was run                        | T1036.007/T1204.002             |
| SYS-200 | Process Create | High     | UAC auto-elevate binary launched outside the normal Explorer prompt flow    | T1548.002                       |
| SYS-201 | Registry Set   | Critical | Per-user %windir% environment variable set via registry                     | T1548.002                       |
| SYS-202 | Image Load     | Critical | UAC auto-elevate binary loaded an unsigned DLL                              | T1548.002/T1574.001/.002        |
| SYS-203 | Registry Set   | Critical | UAC disabled via the EnableLUA registry value                               | T1548.002/T1562.001             |
| SYS-204 | Process Create | Critical | Shell spawned by DllHost.exe at High integrity                              | T1548.002/T1134.001             |

Rule IDs follow no severity ordering; they are assigned sequentially as rules are added. "Event" is the Sysmon Event ID the rule is indexed under (Process Create = 1, Network Connection = 3, Driver Load = 6, Image Load = 7, Remote Thread Created = 8, Process Access = 10, File Create = 11, Registry Create/Delete = 12, Registry Set = 13, Pipe Created = 17, WMI Event = 20).



## 5. The Analyst Console

The console is a single-page, dark-surface application served directly by the backend at the site root, with a live WebSocket feed so every connected analyst sees the same state update in real time.

### 5.1 Incident Queue

The primary view: every incident the engine has correlated, most recent first, filterable between “Triage” (actionable incidents only, i.e. those over the score threshold), “All” (everything the engine is tracking, including sub-threshold activity it has not promoted), and “Closed” (incidents an analyst has already resolved -- see Section 5.9). Each row shows the derived title, severity, host, cumulative score, the process chain, and ATT&CK technique chips at a glance, before any drill-down. An incident that matches one of the correlation chains from Section 3.8 (ransomware, credential-theft campaign, Office-to-PowerShell) additionally carries a filled classification badge next to its title, repeated on the incident's full page.

### 5.2 Incident Detail Views

Expanding an incident reveals its behavior-profile narrative, then three interchangeable views of the same underlying data:

- **List** -- every detection with its full forensic context: process, parent, user, integrity level, hashes (with a one-click VirusTotal lookup), and, for file-write detections, the exact matched file path.
- **Timeline** -- detections placed in sequence with the real elapsed time between steps; clicking a node opens a floating forensic detail popup.
- **Process tree** -- the complete branching tree the incident spans, not just the chain to the triggering process. Nodes that fired a detection are colour-coded by severity; benign context processes are shown hollow so the whole tree's shape is visible, not only the flagged parts.

### 5.3 Explore View

The inline detail views are deliberately compact so they fit beside the detection list, which is too little room for a wide tree, a long timeline, or a long log list. “Explore” hands the same incident to a dedicated full-screen page (`/incident/{id}/explore`) with three tabs switchable in place, each a click away from the other: process tree, timeline, and a plain scrollable log. The tree and timeline gain click-and-drag panning, scroll-wheel zoom centred on the cursor, a zoom-to-fit control, and keyboard shortcuts (+ / - / 0). Clicking a node or a timeline entry opens the same forensic detail panel used inline.

### 5.4 Search

One search box combines free text with field filters, mixed freely in a single query -- for example, `powershell severity:critical host:fin-ws`. Free text matches anywhere an analyst might remember something about an incident: a process name, a command-line fragment, a hash, a rule ID, the title. Field filters narrow precisely by substring match, not exact match, so `host:fin-ws` finds `FIN-WS-07`.

| Filter                  | Matches                                               |
|-------------------------|-------------------------------------------------------|
| <code>host:</code>      | Substring match against the incident's host.          |
| <code>severity:</code>  | Exact match: info, low, medium, high, or critical.    |
| <code>technique:</code> | Matches an ATT&CK technique ID, exact or as a prefix. |



| Filter        | Matches                                                                   |
|---------------|---------------------------------------------------------------------------|
| rule:         | Matches a detection rule ID.                                              |
| user:         | Matches the forensic user field on a detection.                           |
| command_line: | Scoped substring match against a detection's command line only.           |
| actionable:   | true/false -- restricts to incidents over (or under) the score threshold. |

## 5.5 IOC Enrichment

IP addresses, domains, and file hashes can be checked on demand against AbuseIPDB (IP reputation) and VirusTotal (IP, domain, and file-hash reputation). Results are cached for an hour. Every provider degrades gracefully to “unavailable” with no API key configured -- enrichment never blocks the rest of the console from working. Private IP ranges are never sent to a third party, and the strongest available hash (SHA-256 over SHA-1 over MD5) is always preferred, since MD5 collisions are cheap enough for malware authors to produce deliberately.

## 5.6 ATT&CK Technique Reference

Every technique chip shown anywhere in the console is clickable and opens its official MITRE description -- name, tactics, and full text -- from a local copy of the ATT&CK STIX dataset, with a link out to the ATT&CK site. No lookup leaves the console for an analyst who does not have T1003.001 memorized.

## 5.7 Analyst Notes

Each incident's full-page view carries a free-text analyst note, capped at 500 words -- a summary, not a report -- with autosave on the incident page and a live word counter.

## 5.8 PDF Reports & STIX Export

Every incident can be downloaded as a self-contained PDF report, suitable for attaching to a ticket or handing to an incident-response lead. It is not a screenshot of the console -- it is the investigation written down: the behavior-profile narrative, the process chain, every detection with its forensics, and the incident's key indicators. Generated server-side with reportlab, requiring no headless browser in the deployment image. The button beside it, “Download STIX bundle”, exports the same incident as a machine-readable STIX 2.1 bundle instead -- see Section 3.10 for exactly what it contains.

## 5.9 Incident Triage

Every incident carries a status: open, closed, or a verdict such as false positive or benign, set from a “Set verdict” menu on the incident card and cleared with a “Close” action. Any state can transition to any other -- a closed incident can be reopened if new evidence surfaces -- and there is no workflow enforced beyond that; the analyst is the one doing the triage, not the console. Closed incidents drop out of the default queue view and are reachable from the “Closed” filter tab (Section 5.1), so a shift's queue only ever shows what still needs a decision.

Once at least one incident has been marked a false positive, later open incidents that resemble it closely enough gain a dashed “probable noise” badge next to the status badge, naming the similarity percentage -- see Section 3.12 for the scoring behind it.



## 5.10 Sigma Rule Import

The same settings menu that holds the theme toggle also holds “Import Sigma rules”, which opens a file picker accepting one or more .yaml/.yml files. Selecting files immediately uploads them to `POST /admin/rules/import-sigma`; a report modal then lists every document that was accepted (with its generated rule ID) and every one that was rejected (with the specific reason), so a partially-successful batch is never a silent partial success. See Section 3.9 for exactly what subset of Sigma is supported.

## 5.11 Theme & Database Reset

A settings menu in the console header holds a light/dark theme toggle and one destructive action: wiping the database. The reset is confirmed with a dialog, since there is no undo -- every detection and incident is permanently removed, on every connected console, via the same WebSocket broadcast that delivers live updates.

## 5.12 Live WebSocket Feed

The console holds a persistent WebSocket connection and reconnects automatically on drop, so it can be left open on a wall display indefinitely. New detections, incident updates, and database resets are pushed to every connected client the instant they happen -- there is no polling and no manual refresh.

## 5.13 ATT&CK Coverage Download

The same settings menu holds “Download ATT&CK coverage”, a direct link to `GET /attack/coverage/navigator` -- no upload step, no report modal, just a MITRE ATT&CK Navigator layer file ready to open at the public Navigator. See Section 3.11 for what it contains and how it degrades when the full technique index is not present.

## 5.14 Stats Dashboard

A fourth console view, linked from the header (`GET /dashboard`), independent of any one incident: incidents per day over a 7/14/30/90-day range with every day in the window shown, including the zero-count ones, so a quiet stretch reads as a flat bar rather than a gap in the series; severity distribution and triage status across every incident on record; and the top 10 rules and top 10 ATT&CK techniques by detection count, each technique linking directly to its MITRE page. Backed by `GET /stats` (backend/engine/stats.py), which reads a narrow column set from SQLite and tallies it in Python rather than expressing a “group by calendar day” in SQL. Charted as plain HTML/CSS bars -- no charting library, matching the rest of the frontend's zero-dependency approach.





## 6. REST API Reference

The full interactive OpenAPI documentation is served at `/api/docs` (Swagger UI) and `/api/redoc` (ReDoc) on a running instance. The table below is a static summary of every route.

| Method | Path                                    | Description                                                                                 |
|--------|-----------------------------------------|---------------------------------------------------------------------------------------------|
| POST   | <code>/ingest</code>                    | Accept one normalized Sysmon/Winlogbeat event; runs it through the full pipeline.           |
| GET    | <code>/incidents</code>                 | List incidents, most recent first.                                                          |
| GET    | <code>/incidents/{id}</code>            | Full incident detail: detections, process tree, forensics.                                  |
| PUT    | <code>/incidents/{id}/status</code>     | Close an incident, mark it a false positive, or reopen it.                                  |
| PUT    | <code>/incidents/{id}/notes</code>      | Set the incident's analyst note (500-word limit).                                           |
| DELETE | <code>/incidents/{id}/notes</code>      | Clear the incident's analyst note.                                                          |
| GET    | <code>/incidents/{id}/profile</code>    | Kill-chain narrative summary for the incident.                                              |
| GET    | <code>/incidents/{id}/report</code>     | Generate and download the incident's PDF report.                                            |
| GET    | <code>/incidents/{id}/stix</code>       | Generate and download the incident as a STIX 2.1 bundle.                                    |
| GET    | <code>/detections</code>                | List raw detections, most recent first.                                                     |
| GET    | <code>/search</code>                    | Free-text and field-filtered search across incidents.                                       |
| GET    | <code>/attack/coverage</code>           | Rule-coverage report: rule/detector count per ATT&CK technique.                             |
| GET    | <code>/attack/coverage/navigator</code> | The coverage report as a downloadable MITRE ATT&CK Navigator layer.                         |
| GET    | <code>/attack/{technique_id}</code>     | MITRE ATT&CK technique description, from the local dataset.                                 |
| GET    | <code>/stats</code>                     | Dashboard aggregates: incidents/day, severity mix, triage totals, top rules and techniques. |
| GET    | <code>/enrich</code>                    | On-demand reputation lookup for an IP, domain, or hash.                                     |
| DELETE | <code>/admin/database</code>            | Wipe all detections and incidents; reset the live engine.                                   |
| POST   | <code>/admin/rules/import-sigma</code>  | Convert and load one or more Sigma YAML files; live immediately.                            |
| GET    | <code>/health</code>                    | Liveness/readiness probe: rule count, tracked processes.                                    |
| GET    | <code>/metrics</code>                   | Prometheus text-exposition scrape target: request, ingest, and detection counters.          |
| WS     | <code>/ws</code>                        | Live event stream: new detections, incident updates, resets.                                |
| GET    | <code>/</code>                          | The analyst console (single page app).                                                      |
| GET    | <code>/incident/{id}</code>             | Full-page view of one incident, with notes editor.                                          |
| GET    | <code>/incident/{id}/explore</code>     | Full-screen Explore view: process tree, timeline, or logs, picked by tab.                   |
| GET    | <code>/incident/{id}/tree</code>        | Alias for <code>/explore?view=tree</code> , kept for old links.                             |
| GET    | <code>/dashboard</code>                 | The stats dashboard page: incident trends and top rules/techniques, charted.                |



## 7. Configuration Reference

Every tunable lives in one settings object and can be overridden from the environment or a `.env` file, without touching code. All variables are prefixed `HUNTER_`.

| Variable                            | Default                            | Purpose                                                                            |
|-------------------------------------|------------------------------------|------------------------------------------------------------------------------------|
| HUNTER_DB_URL                       | sqlite+aiosqlite:///data/hunter.db | Database connection string.                                                        |
| HUNTER_API_KEY                      | (unset)                            | Shared secret for the JSON API. Unset = no auth (trusted network).                 |
| HUNTER_CORRELATION_WINDOW_MINUTES   | 10                                 | Time window for grouping detections into one incident.                             |
| HUNTER_INCIDENT_SCORE_THRESHOLD     | 12                                 | Cumulative severity score to promote an incident to active.                        |
| HUNTER_PROCESS_TTL_MINUTES          | 120                                | How long an inactive process stays in the in-memory tree.                          |
| HUNTER_BEACON_ENABLED               | true                               | Enable/disable statistical beacon detection.                                       |
| HUNTER_BEACON_MIN_CONNECTIONS       | 6                                  | Callbacks required before a beacon verdict.                                        |
| HUNTER_BEACON_REGULARITY_THRESHOLD  | 0.75                               | Minimum regularity score (0-1) to flag a beacon.                                   |
| HUNTER_BEACON_MIN_INTERVAL_SECONDS  | 5.0                                | Fastest interval considered beacon-like, not streaming.                            |
| HUNTER_BEACON_MAX_INTERVAL_SECONDS  | 3600.0                             | Slowest interval still distinguishable from a scheduled task.                      |
| HUNTER_DISCOVERY_ENABLED            | true                               | Enable/disable reconnaissance-burst detection.                                     |
| HUNTER_DISCOVERY_MIN_DISTINCT       | 4                                  | Distinct recon techniques required to flag a burst.                                |
| HUNTER_SCAN_ENABLED                 | true                               | Enable/disable statistical network-scan detection.                                 |
| HUNTER_SCAN_MIN_DISTINCT_IPS        | 10                                 | Distinct destination IPs from one process required to flag a host sweep.           |
| HUNTER_SCAN_MIN_DISTINCT_PORTS      | 15                                 | Distinct destination ports from one process required to flag a port scan.          |
| HUNTER_NOISE_SIMILARITY_THRESHOLD   | 0.6                                | Similarity score (0-1) an open incident needs to be flagged as probable noise.     |
| HUNTER_ABUSEIPDB_API_KEY            | (unset)                            | Optional key for IP reputation enrichment.                                         |
| HUNTER_VIRUSTOTAL_API_KEY           | (unset)                            | Optional key for hash/IP/domain reputation enrichment.                             |
| HUNTER_ENRICHMENT_CACHE_TTL_SECONDS | 3600                               | How long a reputation lookup is cached.                                            |
| HUNTER_LOG_JSON                     | false                              | One JSON object per log line, for log aggregators, instead of human-readable text. |
| HUNTER_INGEST_RATE_LIMIT_PER_SECOND | 0                                  | Token-bucket rate limit on /ingest per source IP. 0 disables it.                   |
| HUNTER_INGEST_RATE_LIMIT_BURST      | 50                                 | Burst allowance above the steady-state /ingest rate limit.                         |



## 8. Installation & Getting Started

Requires Python 3.11 or newer.

```
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt

python -m alembic upgrade head    # create the database schema
python -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

Console: <http://localhost:8000> · API docs: <http://localhost:8000/api/docs>

### Seeding demo data

```
python scripts/seed_apr.py    # one deep multi-stage intrusion
python scripts/seed_demo.py   # varied incidents across the kill chain
python scripts/seed_rw.py     # a full ransomware chain in one incident
```

### Analysing a real sample

Replay a Sysmon .evtx file -- from a lab VM, or a public corpus such as EVT-X-ATTACK-SAMPLES, where each file exercises one ATT&CK technique:

```
pip install evtx
python scripts/replay_evtx.py --file samples/sysmon_credential_access.evtx
```

### Optional: IOC enrichment API keys

Works with no keys configured (every provider reports unavailable). For live reputation data, add free-tier API keys to a .env file:

```
HUNTER_ABUSEIPDB_API_KEY=...
HUNTER_VIRUSTOTAL_API_KEY=...
```

### Optional: API key (authentication)

The server binds to all interfaces (0.0.0.0) by default, so it is reachable from anywhere on the network it runs on -- fine for a single trusted network, not fine once it is reachable more broadly. Setting HUNTER\_API\_KEY makes every JSON endpoint (not the console's own HTML pages) require a matching X-API-Key header; see backend/api/auth.py.

Generate a random secret with whichever tool is on hand:

```
python -c "import secrets; print(secrets.token_hex(32))" # any platform
openssl rand -hex 32                                # Linux / macOS
```

Then set it in .env:

```
HUNTER_API_KEY=<paste the generated string here>
```

The console detects a 401 on its own and prompts once for the key, then remembers it in the browser -- no other setup needed. Treat the key like a password: do not commit it, and prefer HTTPS (e.g. behind a reverse proxy) if the server is reachable outside a trusted network.

## 9. Testing

```
pip install pytest pytest-asyncio
python -m pytest    # 708 tests
```



The test suite doubles as documentation: each design decision named in Section 11 has a test named after it, and every shipped detection rule is validated against a true positive it must catch and a true negative it must ignore.

## 10. Docker Deployment

```
docker compose up --build
```

A multi-stage build keeps the runtime image to the application and its virtual environment only -- the build tooling is discarded. The container runs as an unprivileged user, applies database migrations automatically on start, and exposes a health check at `/health`.

## 11. Design Decisions

The choices below are what separate this engine from a pattern match over a log file. Each is enforced by an automated test named after it.

### **Correlation keys on ProcessGuid, never PID.**

Windows recycles process IDs aggressively; a PID-keyed tree would graft a malicious child onto whatever unrelated process later inherits its parent's PID. ProcessGuid is unique across reboots and PID reuse.

### **The process tree observes everything, not just detections.**

A malicious process's ancestors are usually entirely benign, so the process that never triggers a rule on its own must still be recorded, or it can never appear as the root of a phishing chain.

### **Incident scoring is non-linear.**

One critical LSASS access outweighs three medium-severity suspicious-path executions. Two high-severity detections read as critical together. An incident can be worse than any single rule that composes it.

### **Beaconing uses median and median absolute deviation, not mean and standard deviation.**

A live C2 session produces outliers constantly. One unusually long gap in an otherwise 60-second beacon wrecks a standard-deviation-based score and loses the channel; the median barely notices. Jitter up to Cobalt Strike's default 37% is still caught.

### **Discovery counts distinct techniques, not raw execution counts.**

A script re-running one command in a loop is volume without variety. An operator running several different reconnaissance commands in quick succession is variety, and only the second pattern is flagged.

### **Enrichment works with no API keys, and never leaks internal data.**

Every provider degrades to "unavailable" without a key rather than failing the request. Private IP ranges are never sent to a third party. The strongest available hash is always preferred over a weaker one.

### **A missing optional data file degrades the ATT&CK coverage report, never crashes it.**

Without the full technique index the coverage report and its Navigator export still run -- they just say so, falling back to ranking only the techniques this project already covers instead of refusing to answer at all.



## 12. Project Layout

```
sysmon-hunter/  
+-- backend/  
| +-- main.py          FastAPI app, lifespan, background sweep, /metrics  
| +-- config.py        all tunables  
| +-- logging_setup.py text/JSON log formatting  
| +-- api/             ingest, detections, incidents, attack, enrich,  
| |                   report, search, notes, admin, ws, serializers,  
| |                   rate_limit, stats  
| +-- engine/          normalizer, rule_loader, matcher, correlator,  
| |                   beacon, discovery, scan, attack, coverage, enrichment,  
| |                   search, report, profile, pipeline, sigma_import,  
| |                   stix_export, noise, metrics, stats  
| +-- models/          schemas, db  
| `-- data/            attack_data.json (ATT&CK technique lookup),  
|                   attack_index.json (full catalog, coverage gaps)  
+-- rules/             157 YAML detection rules, by Event ID, plus  
|                   imported_sigma/ for runtime Sigma imports  
+-- frontend/          console.html, incident.html, tree.html,  
|                   dashboard.html, static/{css,js}  
+-- migrations/        Alembic  
+-- scripts/           seed_apr, seed_demo, seed_rw, seed_full_coverage,  
|                   replay_evtx, fetch_attack  
+-- docs/              screenshots  
`-- tests/             708 tests
```

*Built as a hands-on threat-detection research project. The engine targets a single collector; scaling to many would mean moving the queue to Redis and the store to Postgres, both isolated behind small interfaces so that change touches one file each.*